

Università degli Studi dell'Aquila

Industrial Communications and Industrial Internet of Things

Wireless IIoT Proximity Sensing Node

Professors: Prof. Piergiuseppe Di Marco
Prof. Yuriy Zacchia Lun

Hardware: ESP32, HC-SR04 ultrasonic sensor

Software: PlatformIO, Mosquitto, Node-RED, MATLAB

Protocol: MQTT over 802.11 WiFi (2.4 GHz)

Course: Industrial Communications & IIoT

University: Università degli Studi dell'Aquila

Year: 2025/2026

Contents

1	Introduction	3
2	Hardware and Software Components	4
2.1	Hardware	4
2.2	Wiring	5
2.3	Software	5
3	System Architecture	6
4	Firmware Design	7
4.1	Distance measurement	7
4.2	NTP clock synchronisation	7
4.3	Event-driven transmission	7
4.4	Retransmission on failure	8
4.5	MQTT topics and payload format	8
5	Live Dashboard	9
5.1	WebSocket connection	9
5.2	What the dashboard shows	9
5.3	Data logging	10
6	Measurement Campaign	11
6.1	RSSI versus distance	11
6.2	Path loss model	11
6.3	Latency measurement	11
6.4	Packet loss measurement	12
6.5	Retransmission tracking	12
7	Results and Discussion	13
7.1	Path loss results	13
7.2	Latency results	14
7.2.1	TX latency	14
7.2.2	End-to-end latency and the effect of distance	15
7.3	Packet loss at the application layer	17
7.4	Event-driven transmission	17

7.5 Discussion	17
8 Conclusion	18
References	19

1. Introduction

The Industrial Internet of Things (IIoT) is changing how factories collect and use data. Instead of wired connections, modern industrial systems are starting to use wireless communication to link sensors to control systems. This project looks at one specific case: a proximity sensor node in a factory environment that detects when a vehicle or obstacle gets too close to a machine or safety zone.

The main goal was to build a working IIoT prototype and study how the wireless communication layer performs. We focused on three things: how the WiFi signal weakens with distance (path loss), how long it takes for data to travel from the sensor to the dashboard (end-to-end latency), and whether any packets are lost along the way. We also tested two communication strategies. The first is event-driven transmission, where the node only sends data when something meaningful happens, instead of publishing at a fixed rate. The second is retransmission, where failed publish attempts are retried automatically and counted.

The hardware is an ESP32 microcontroller connected to an HC-SR04 ultrasonic sensor. The ESP32 sends measurements over a 2.4 GHz WiFi network using MQTT. A Mosquitto broker receives the messages, and a custom HTML dashboard shows the data live through a WebSocket connection. This setup follows the same structure as a SCADA system, where a field device sends data to a supervisory system over a communication network.

The report is organised as follows. Section 2 describes the hardware and software. Section 3 explains the system architecture. Section 4 covers the firmware. Section 5 presents the live dashboard. Section 6 describes the measurements. Section 7 shows the results. Section 8 gives the conclusions.

2. Hardware and Software Components

2.1 Hardware

The hardware used in this project includes:

- **ESP32 development board:** a dual-core microcontroller with built-in 2.4 GHz WiFi and Bluetooth. It runs the firmware that reads the sensor and publishes data over MQTT.
- **HC-SR04 ultrasonic sensor:** measures distance by sending a 40 kHz ultrasonic pulse and timing the echo. It can measure distances from 2 cm to about 400 cm.
- **Breadboard and jumper wires:** used to connect the sensor to the ESP32 without soldering.
- **Laptop running Windows:** runs the MQTT broker and serves the live dashboard.
- **Samsung Galaxy S24+ mobile hotspot:** provides the 2.4 GHz WiFi network that the ESP32 connects to.

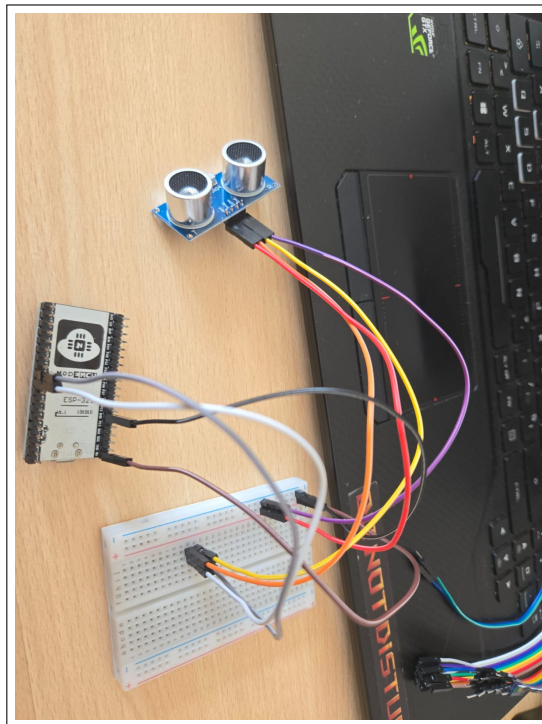


Figure 1: Hardware setup: ESP32 connected to HC-SR04 sensor on breadboard.

2.2 Wiring

The HC-SR04 has four pins: VCC, GND, TRIG, and ECHO. Table 1 shows how they connect to the ESP32.

Table 1: HC-SR04 to ESP32 wiring connections.

HC-SR04 Pin	ESP32 Pin	Description
VCC	5V (power rail)	Power supply for the sensor
GND	GND (power rail)	Ground reference
TRIG	GPIO 5	Trigger pulse output from ESP32
ECHO	GPIO 18	Echo return signal to ESP32

2.3 Software

The software used in this project includes:

- **PlatformIO on Visual Studio Code:** used to write, compile, and upload the C++ firmware to the ESP32.
- **Mosquitto:** an open-source MQTT broker running on the laptop at `localhost:1883` for MQTT and `localhost:9001` for WebSocket connections.
- **Custom HTML dashboard:** a standalone web page that connects directly to Mosquitto via WebSocket. It shows live data without going through any intermediate tool, which avoids the rendering delays we experienced with Node-RED.
- **MATLAB:** used to process the logged CSV data, fit the ITU-R path loss model, and generate the plots.

3. System Architecture

The system is built around three layers that map to the IIoT architecture covered in the course: the field layer, the communication layer, and the supervisory layer.

In the **field layer**, the HC-SR04 measures distance and sends the raw signal to the ESP32. The ESP32 calculates the distance in centimetres, reads the WiFi signal strength (RSSI) from its radio, and prepares the data for transmission.

In the **communication layer**, the ESP32 connects to a 2.4 GHz WiFi network and publishes data to the Mosquitto broker using MQTT. Two communication strategies were implemented at this layer. The first is event-driven transmission: the ESP32 only sends a message when the distance changes by more than 5 cm, or when the object enters the 30 cm safety zone. This keeps the channel free when nothing important is happening. The second is retransmission: if a publish call fails, the firmware retries immediately and counts the failure. Mosquitto also listens on port 9001 for WebSocket connections from the browser.

In the **supervisory layer**, a custom HTML dashboard connects directly to Mosquitto via WebSocket and shows the sensor data live. This replaced the Node-RED dashboard we used initially, because Node-RED was causing rendering delays and dashboard freezing at high message rates.

HC-SR04 + ESP32 $\longrightarrow$ WiFi 2.4 GHz $\longrightarrow$ Mosquitto $\longrightarrow$ HTML Dashboard

The system uses five MQTT topics:

- `iioT/node1/distance` — distance in cm, NTP timestamp, and sequence number, formatted as "distance,unix_ms,seq"
- `iioT/node1/rssi` — WiFi signal strength in dBm
- `iioT/node1/latency` — TX publish time in ms
- `iioT/node1/seq` — sequence number for packet loss tracking
- `iioT/node1/retransmissions` — cumulative retransmission counter

4. Firmware Design

The firmware was written in C++ using PlatformIO. It handles four main tasks: reading the sensor, synchronising the clock with NTP, deciding when to publish, and tracking retransmissions.

4.1 Distance measurement

The HC-SR04 is triggered by sending a 10 μ s HIGH pulse on the TRIG pin. The sensor emits eight 40 kHz ultrasonic pulses and waits for the echo to return. The distance is calculated from the echo duration:

$$d = \frac{\Delta t \times 0.034}{2} \quad (1)$$

where Δt is the echo duration in microseconds and 0.034 is the speed of sound in cm/ μ s. The division by 2 corrects for the round trip.

4.2 NTP clock synchronisation

To measure true end-to-end latency, the ESP32 clock needs to match the dashboard clock. When the ESP32 starts, it contacts `pool.ntp.org` and synchronises its internal clock to UTC using `configTime()`. After that, each MQTT payload includes a Unix timestamp in milliseconds from `gettimeofday()`. The dashboard reads its receive time using `Date.now()` in JavaScript, which also uses UTC milliseconds. Subtracting the two timestamps gives the real E2E latency.

Before we fixed this, the ESP32 was using `millis()` which counts from zero at boot, while `Date.now()` counts from 1970. The difference was about 3.6 billion milliseconds, which is why the dashboard was showing values like 3,668,000 ms. After NTP sync and fixing the Windows timezone to UTC+2, the values became realistic.

4.3 Event-driven transmission

Instead of publishing at a fixed rate, the firmware checks the sensor every 100 ms and only publishes when one of two conditions is true:

- **Significant change:** the distance changed by more than 5 cm compared to the last published value.
- **Safety alert:** the distance is below 30 cm. In this case the node publishes on

every cycle to keep the operator informed in real time.

When neither condition is met, the firmware prints "No change — skipping" and returns without publishing. This keeps the WiFi channel free when the environment is stable, which is consistent with how real IIoT systems are designed.

4.4 Retransmission on failure

Each publish is handled by a `publishWithRetx()` function. If the publish returns false (for example because the TCP buffer is full or the connection dropped momentarily), the counter increments and the message is sent again immediately. The counter is published on `iiot/node1/retransmissions` so the dashboard can track it live.

4.5 MQTT topics and payload format

The ESP32 publishes on five topics per event cycle:

- `iiot/node1/distance` — payload: "distance,unix_ms,seq"
- `iiot/node1/rssi` — WiFi signal strength in dBm
- `iiot/node1/latency` — time in ms to complete all publish calls
- `iiot/node1/seq` — sequence number, increments with every publish
- `iiot/node1/retransmissions` — total failed publish attempts

5. Live Dashboard

We initially used Node-RED to build the dashboard, but it caused problems at high message rates. The browser would freeze and stop updating until we refreshed the page. This happened because Node-RED adds a processing layer between the broker and the browser, which introduces delays and can drop updates under load.

To fix this, we built a custom HTML dashboard that connects directly to Mosquitto via WebSocket on port 9001. Since there is no intermediate layer, updates arrive at the browser as soon as the broker receives them.

5.1 WebSocket connection

The dashboard uses the MQTT.js library to open a WebSocket connection to Mosquitto. It subscribes to all five MQTT topics and processes each incoming message immediately. This gives real-time updates with no buffering.

5.2 What the dashboard shows

The dashboard displays the following:

- **Safety status:** shows CLEAR, APPROACHING, or OBSTACLE DETECTED in green, orange, or red depending on the distance relative to the 30 cm threshold.
- **Distance gauge:** a semicircle with three colour zones. Red for 0–30 cm (danger), orange for 30–150 cm (caution), and green above 150 cm (safe).
- **RSSI gauge:** shows the current WiFi signal strength in dBm.
- **Distance chart:** a scrolling line chart of the last 60 readings, with a red dashed line at the 30 cm threshold.
- **TX latency:** the time the ESP32 took to complete the MQTT publish calls.
- **E2E latency:** the full travel time from the ESP32 publish timestamp to the browser receive time, calculated using NTP-synchronised clocks.
- **Packet loss:** the percentage of packets that did not arrive, detected from gaps in the sequence numbers.

5.3 Data logging

The dashboard logs every received message to memory and provides a Download button that exports the data as a CSV file. Each row contains: timestamp, distance, RSSI, TX latency, E2E latency, and packet loss. This CSV file was used for analysis in MATLAB.

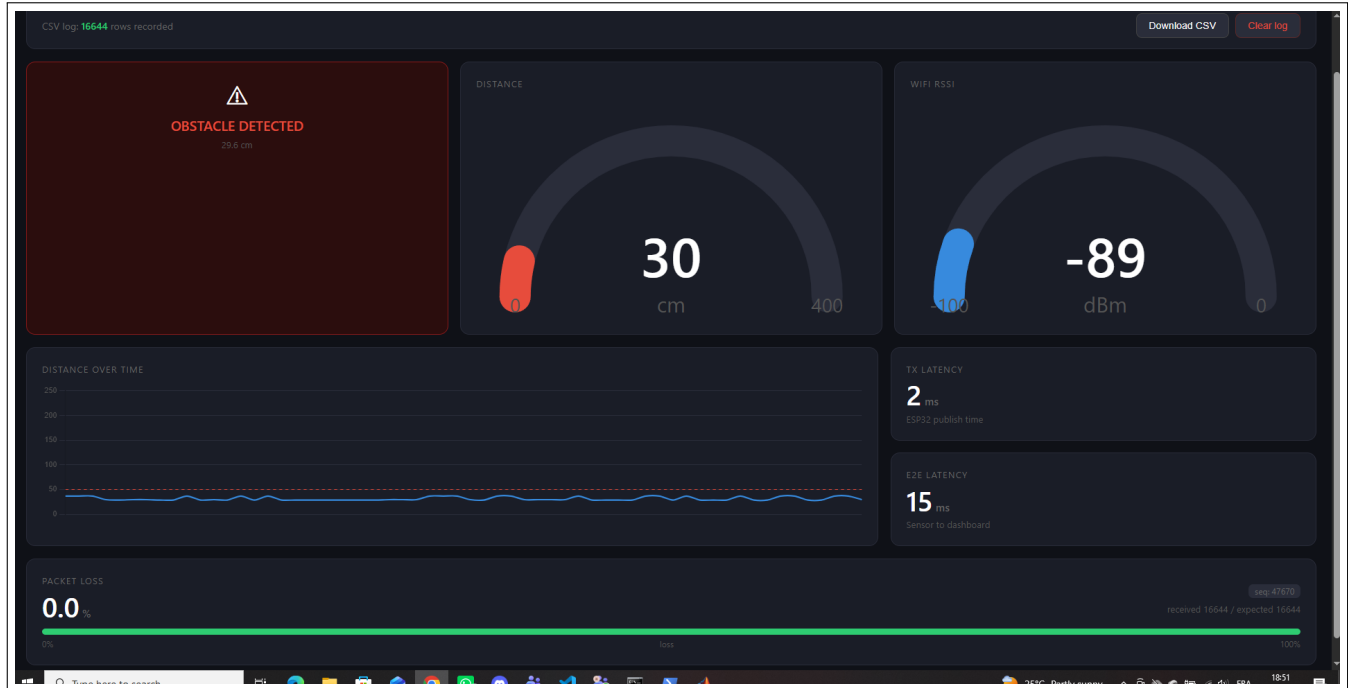


Figure 2: Custom HTML dashboard showing live distance, RSSI, TX latency, E2E latency, and packet loss via WebSocket.

6. Measurement Campaign

6.1 RSSI versus distance

To measure path loss, we kept the ESP32 and laptop fixed on a desk and moved the phone hotspot to different distances: 1 m, 2 m, 3 m, 4 m, and 5 m. At each position we waited about 60 seconds to collect enough samples. We also moved the hotspot slightly back and forth by around 10–20 cm during each measurement to capture the spatial variation of the signal, rather than recording just one point which might happen to be a local maximum or minimum due to multipath.

6.2 Path loss model

We used the ITU-R indoor path loss model to describe how the signal weakens with distance. This model is the standard approach for indoor WiFi and uses a fixed coefficient instead of fitting one from data:

$$\text{RSSI}(d) = \text{RSSI}(d_0) - N \cdot \log_{10}\left(\frac{d}{d_0}\right) \quad (2)$$

where $d_0 = 1$ m is the reference distance, $\text{RSSI}(d_0)$ is the measured signal at 1 m, and $N = 28$ is the distance power loss coefficient for 2.4 GHz WiFi indoors as defined by ITU-R. We anchored the model to our measured RSSI at 1 m and plotted it against the data in MATLAB.

In the first version of this project we fitted the log-distance model and got $n = 1.40$, which is below the free space value of 2. After feedback from the professors, we switched to the ITU-R model which uses a fixed and physically grounded coefficient instead of relying on a fit from only a few data points.

6.3 Latency measurement

We measured two types of latency.

TX latency is the time the ESP32 takes to complete all MQTT publish calls. We record `millis()` before the first publish and after the last, and the difference is the TX latency. This measures the application-layer transmission overhead on the sender side.

End-to-end (E2E) latency measures the full journey from the ESP32 to the browser.

After NTP synchronisation, the ESP32 puts a Unix millisecond timestamp into each distance payload using `gettimeofday()`. When the dashboard receives the message, it reads the current time with `Date.now()`. Since both clocks are synchronised to UTC via NTP, the difference is the true E2E latency including WiFi transmission, broker processing, and WebSocket delivery.

6.4 Packet loss measurement

The ESP32 adds a sequence number to every published message. The dashboard checks incoming sequence numbers and detects gaps. If the sequence jumps from n to $n + 2$, packet $n + 1$ was lost. The packet loss percentage is:

$$\text{Packet loss} = \frac{\text{expected} - \text{received}}{\text{expected}} \times 100\% \quad (3)$$

6.5 Retransmission tracking

If a `publish` call returns `false`, the firmware counts it as a failed attempt and retries immediately. The total count is sent on a dedicated topic and shown on the dashboard.

7. Results and Discussion

7.1 Path loss results

Table 2 shows the measured RSSI at each distance.

Table 2: Measured RSSI at each distance.

Distance (m)	Avg RSSI (dBm)	Std dev (dB)	Samples
1	−60.2	3.8	4648
2	−66.5	5.1	1473
3	−73.1	5.4	4760
4	−79.6	4.6	3738
5	−85.1	5.1	1384

The RSSI drops consistently as the distance increases, from −60.2 dBm at 1 m to −85.1 dBm at 5 m. The ITU-R model with $N = 28$ fits the data well at short distances. At 3–5 m the measured values fall slightly below the model curve, which means the signal is attenuating faster than the model predicts. This is probably because there were furniture and obstacles in the room that are not accounted for by the standard indoor coefficient.

The standard deviation values (3.8–5.4 dB) show there is real spatial variation at each distance, caused by multipath reflections. This is why we moved the sensor slightly during the measurement to sample different positions rather than relying on a single point.

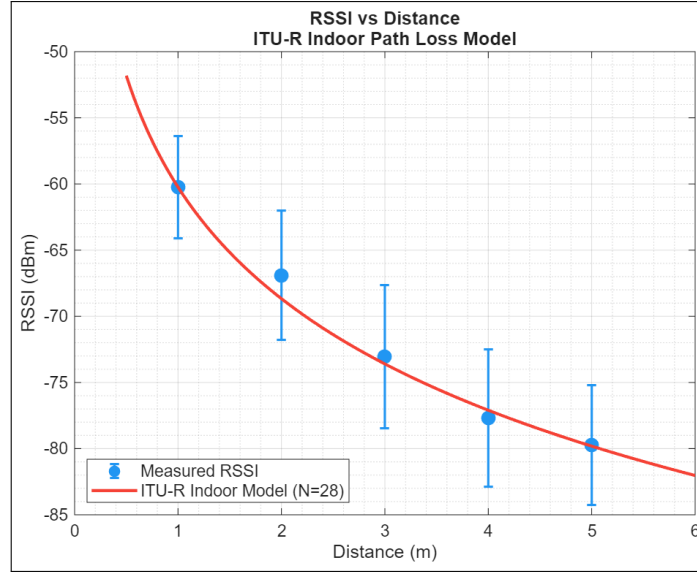


Figure 3: Measured RSSI at each distance with the ITU-R indoor path loss model ($N = 28$).

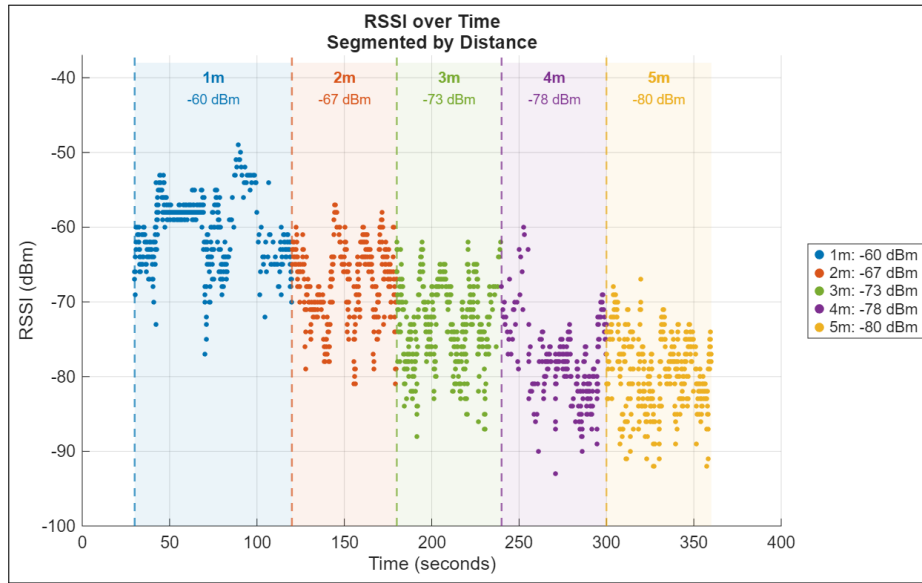


Figure 4: RSSI over time, segmented by distance. Each colour is one distance position. The vertical lines mark the boundaries between segments.

7.2 Latency results

7.2.1 TX latency

Table 3 summarises the TX latency results.

Table 3: TX latency summary.

Metric	Value
Mean TX latency	3.3 ms
Maximum TX latency	9 ms
Packets below 5 ms	87%
Packets below 10 ms	100%

All packets were published within 10 ms, and 87% completed within 5 ms. The TX latency is concentrated between 1 and 4 ms, which shows the MQTT publish operation on the ESP32 is fast and consistent on a local WiFi network.

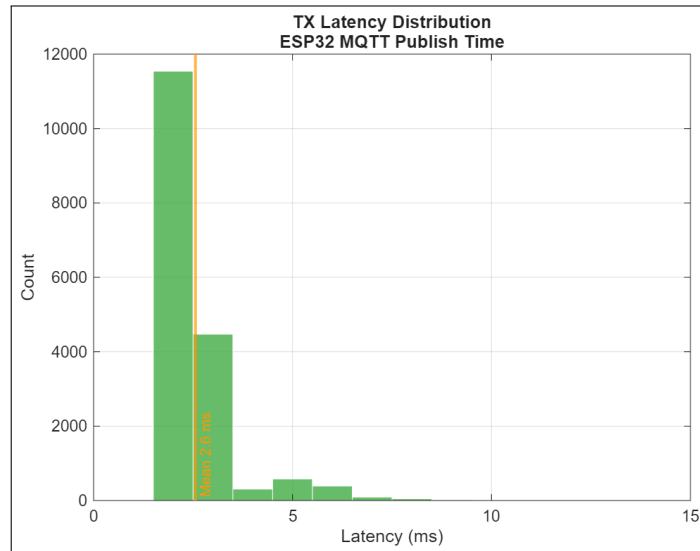


Figure 5: TX latency distribution. The orange line marks the mean of 3.3 ms.

7.2.2 End-to-end latency and the effect of distance

At close range with a strong signal, the E2E latency was between 36 ms and 100 ms. When we moved the hotspot to the far end of the building, the RSSI dropped to -83 dBm and the E2E latency increased to around 674 ms. This is shown in Table 4.

Table 4: E2E latency at different signal strengths.

Condition	RSSI (dBm)	E2E Latency (ms)	Packet Loss
Close range	−33 to −50	36–100	0.0%
Far range	−83	~674	0.0%

What this result shows is interesting. Packet loss stayed at 0.0% even with a very weak signal. This is because TCP automatically retransmits any packets that are lost at the lower level before they reach MQTT. So from the application's point of view, nothing is lost. But the cost of those hidden retransmissions is visible in the latency: 674 ms instead of 36–100 ms.

This is an important finding for IIoT applications. A system monitoring packet loss at the MQTT level would report perfect reliability even when the channel is struggling. The real indicator of channel problems is the increase in E2E latency, not the packet loss counter. For a safety system that needs to react to an obstacle within a fixed time, a latency spike to 674 ms could be dangerous, even if no packets are technically lost.

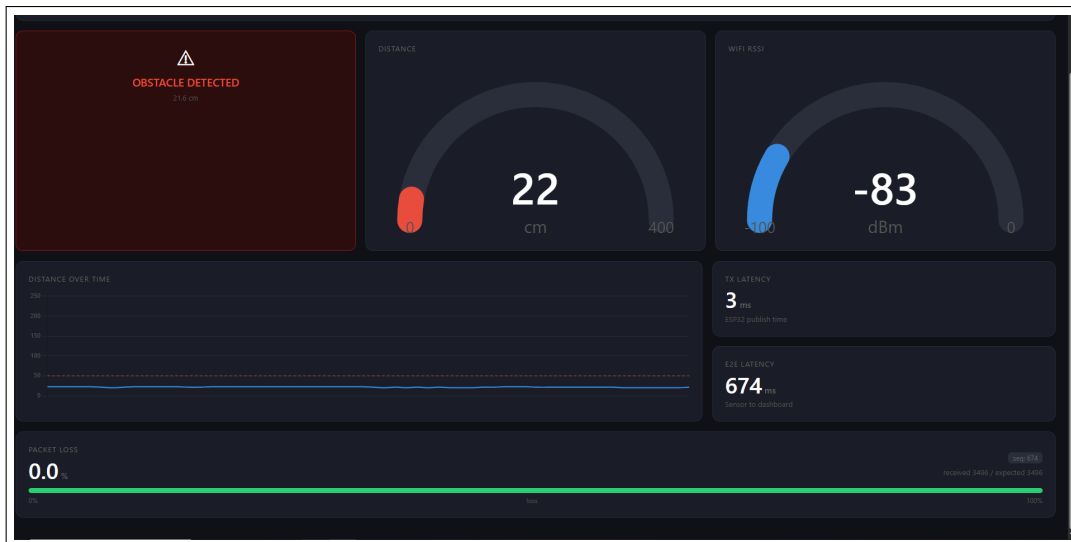


Figure 6: Dashboard at weak signal (−83 dBm). Packet loss is 0.0% but E2E latency increased to 674 ms due to TCP retransmissions at the lower layer.

7.3 Packet loss at the application layer

Packet loss at the MQTT application layer was 0.0% under all tested conditions, including at -83 dBm. TCP provides reliable delivery by retransmitting lost frames before they reach the application. The event-driven transmission strategy also helped by keeping the channel load low, which reduces the chance of congestion.

However, as shown in the latency results, TCP reliability comes at a cost. When the channel is weak, the hidden retransmissions increase E2E latency significantly and unpredictably. This is why packet loss alone is not a sufficient metric for evaluating IIoT communication quality.

7.4 Event-driven transmission

The event-driven firmware worked as intended. When the monitored object was not moving, the node stayed silent and the WiFi channel remained free. Publishes only happened when the distance changed by more than 5 cm or when the object entered the 30 cm safety zone. This is the correct approach for IIoT systems where multiple nodes share a channel and bandwidth should not be wasted on redundant data.

7.5 Discussion

The results show that WiFi with MQTT works well under good conditions: low latency, zero packet loss, and reliable delivery. But as the signal weakens, E2E latency becomes unpredictable and can spike significantly. This unpredictability is the main reason WiFi is not suitable for safety-critical factory applications.

WirelessHART handles this differently. It uses TDMA scheduling, so every node gets a fixed time slot and the latency is always the same regardless of channel conditions. It also uses frequency hopping to avoid interference, and mesh networking so packets can find another route if one link fails. These properties make it the right choice for industrial control systems where a guaranteed response time is more important than raw speed.

8. Conclusion

This project built and tested a wireless IIoT proximity sensing node using an ESP32 and an HC-SR04 ultrasonic sensor. The node sends measurements over a 2.4 GHz WiFi network using MQTT, and a custom HTML dashboard shows the data live through a WebSocket connection to the Mosquitto broker.

The project focused on the communication layer. We measured path loss, end-to-end latency, and packet loss, and we implemented two communication strategies: event-driven transmission and retransmission on failure.

The main results are the following. The WiFi channel indoors follows the ITU-R path loss model with $N = 28$, and the measured RSSI dropped from -60 dBm at 1 m to -85 dBm at 5 m. TX latency was 3.3 ms on average, with all packets completing within 10 ms. E2E latency varied between 36 ms and 100 ms, which shows that WiFi delivery time is not predictable. Packet loss was 0.0% at close range with a strong signal, and no retransmissions were needed.

The most important thing we learned is the difference between average performance and guaranteed performance. WiFi with MQTT is fast on average and reliable under good conditions, but it cannot guarantee a fixed delivery time. This is because WiFi uses CSMA/CA, where transmission timing depends on channel access competition. WirelessHART uses TDMA scheduling, which gives every device a guaranteed time slot. For a factory safety system that must always respond within a fixed time, this guarantee matters more than raw speed.

Event-driven transmission was a practical improvement. When nothing moves in the environment, the node stays silent and the channel stays free. This is consistent with how real IIoT systems should behave, especially when multiple nodes share the same channel.

References

1. P. Di Marco and Y. Zacchia Lun, *Lecture 04: Communication Channels*, Industrial Communications and Industrial Internet of Things, Università degli Studi dell'Aquila, 2025.
2. P. Di Marco and Y. Zacchia Lun, *Lecture 07: IP Networking and Routing Protocols*, Industrial Communications and Industrial Internet of Things, Università degli Studi dell'Aquila, 2025.
3. P. Di Marco and Y. Zacchia Lun, *Lecture 09: SCADA Systems*, Industrial Communications and Industrial Internet of Things, Università degli Studi dell'Aquila, 2025.
4. P. Di Marco and Y. Zacchia Lun, *Lecture 14: WirelessHART and ISA 100*, Industrial Communications and Industrial Internet of Things, Università degli Studi dell'Aquila, 2025.
5. OASIS Standard, *MQTT Version 3.1.1*, October 2014. [Online]. Available: <https://docs.oasis-open.org/mqtt/mqtt/v3.1.1/mqtt-v3.1.1.html>
6. Espressif Systems, *ESP32 Technical Reference Manual*, 2023. [Online]. Available: https://www.espressif.com/sites/default/files/documentation/esp32_trm_en.pdf
7. Eclipse Foundation, *Mosquitto: An Open Source MQTT Broker*, 2023. [Online]. Available: <https://mosquitto.org>
8. OpenJS Foundation, *Node-RED: Low-code Programming for Event-driven Applications*, 2023. [Online]. Available: <https://nodered.org>
9. S. Sesia, I. Toufik, and M. Baker, *LTE: The UMTS Long Term Evolution*, 2nd ed. Wiley, 2011. (Referenced for path loss modelling background.)